

HBC RADIO

ATAK Plugin — Interface Control Document

Ham Binary CoT over Distance-Vector Mesh Radio

Version 1.1 | KE8TQB | github.com/ke8tqb/HBC-ATAK-Plugin

Table of Contents

1. Overview
2. System Architecture
3. Physical Layer (Audio Modems)
 - 3.1 AFSK1200 / AX.25
 - 3.2 OFDM COFDMTV (rattlegram)
 - 3.3 Mercury HF (FreeDV DATAC4)
 - 3.4 Channel Access (CSMA) — taking turns on the air
4. Mesh Network Layer
 - 4.1 Common Packet Header
 - 4.2 Callsign Encoding (ITA2, 5 bytes)
 - 4.3 Packet Type 0x01 — Announce
 - 4.4 Packet Type 0x02 — Broadcast
 - 4.5 Packet Type 0x03 — Direct Message
 - 4.6 Packet Type 0x04 — Acknowledgment
 - 4.7 Route Discovery and Maintenance
 - 4.8 Loop Prevention
 - 4.9 Reliability and Retries
5. Application Layer (HBC Protocol v1.6)
 - 5.1 HBC Header
 - 5.2 Message Modes and Sizes
 - 5.3 Deterministic UIDs
 - 5.4 Direct-Message Tags and Receipts
 - 5.5 Contact Publication
6. User Interface Reference (Screenshots)
7. Timing Parameters and Constants
8. Compatibility and Constraints
9. Glossary
10. References

Appendix A — GitHub Repository File Structure

1. Overview

HBC Radio is a plugin (an add-on) for ATAK, the Android Team Awareness Kit mapping app. It lets ATAK devices share map information — positions, markers, chat messages, and alerts — using ordinary handheld ham radios instead of the internet or a cell network. The phone plays specially coded sound into the radio; another phone hears that sound on its radio and turns it back into map data.

The system is built in three layers, like a stack of responsibilities. Each layer is specified in its own section of this document:

- **Physical layer (Section 3)** — the “voice” of the system: one of three selectable audio modems that turn bytes into sound and back.
- **Mesh network layer (Section 4)** — the “postal service”: it figures out who can hear whom and can pass a message through a middle station to reach someone out of range. Based on a published distance-vector routing design (see References).
- **Application layer (Section 5)** — the “language”: the HBC Protocol, which squeezes a map event into a few dozen bytes so it can be sent quickly over a slow radio link.

In plain terms: HBC (the language) writes a very short letter, the mesh (the postal service) addresses it and picks the route, and the modem (the voice) reads it out loud over the radio.

NOTE: Everything is transmitted unencrypted (in the clear), as required by amateur-radio rules (FCC Part 97). The operator’s callsign is inside every transmission, which satisfies station-identification requirements.

2. System Architecture

The path one message takes from your screen to the other station’s screen:

ATAK map event → HBC encoder (shrinks it to bytes) → MeshRouter (adds addressing) → modem (turns bytes into audio) → radio → ...air... → radio → modem → MeshRouter → HBC decoder → ATAK map

On receive, the mesh layer decides what to do with each frame: deliver it to this device, forward it toward its destination, rebroadcast it (announces), or ignore it.

Layer	Code that implements it	How things are addressed
Application	HbcEncoder / HbcDecoder	Callsign inside the HBC header; predictable UIDs (section 5.3)

Network	MeshRouter	Origin / Transmitter / Destination / Next-hop callsigns
Physical	AudioModem, OfdmModem, MercuryModem	None — just a byte pipe (AFSK adds a fixed AX.25 wrapper for station ID)

3. Physical Layer (Audio Modems)

A modem converts bytes into sound and sound back into bytes. HBC Radio offers three, each suited to different radios and conditions. All three carry the exact same mesh frame, but they are not compatible with each other — **every station on a channel must select the same modem** (Figure 3).

Parameter	AFSK1200	OFDM COFDMTV	Mercury HF
Waveform	Bell 202 AFSK, 1200 baud	QPSK OFDM (rattlegram “short”), 1600 Hz BW	FreeDV DATAC4 raw-data OFDM (codec2)
Sample rate	48000 Hz	48000 Hz	8000 Hz
Payload capacity per burst	Practical AX.25 limits (>200 B)	169 B	52 B (54 B frame minus framing)
Error protection	HDLC CRC-16 (detect only)	Polar code + CRC-32	LDPC + CRC-16
Best for	VHF/UHF FM; compatible with classic packet radio	VHF/UHF FM; survives phone audio processing	HF SSB; works at very weak signal levels

3.1 AFSK1200 / AX.25

The classic 1980s packet-radio sound (two alternating tones, 1200 bits per second). Mesh frames ride inside a single AX.25 “UI” frame. The AX.25 addresses are only wrapping paper — they are not used for routing: destination is always HBC, source is the operator’s ham callsign (which doubles as the legal station ID), and no digipeater path is used. Each transmission is rendered in advance as: lead-in silence (150 ms) → optional VOX leader tone → HDLC flag preamble (“TX Dwell,” default 500 ms) → the frame → tail silence (250 ms).

3.2 OFDM COFDMTV (rattlegram)

A modern, robust waveform that packs data onto many small carriers at once. Framing inside the 170-byte block: byte 0 = mesh frame length (1–169), bytes 1..n = the mesh frame, zero-padded. The operator callsign also rides in the waveform’s own metadata header. Audio carrier is 1500 Hz; the VOX leader is made of noise symbols (~180 ms each).

3.3 Mercury HF (FreeDV DATAC4)

Designed for long-range HF radio, where signals are weak and distorted. The burst layout matches the Mercury HF modem exactly: preamble → one DATAC4 data frame with a CRC-16 checksum in its last two bytes → postamble, with 100 ms of silence on each end. Framing inside the 54 usable bytes: byte 0 = length, byte 1 = the letter 'H' (a marker that says “this is HBC traffic”), bytes 2..n = the mesh frame, zero-padded.

NOTE: This speaks Mercury’s *waveform*, not Mercury’s higher-level ARQ data-link protocol. A desktop Mercury installation can demodulate these bursts but will not pass them to its TCP data port.

3.4 Channel Access (CSMA) — taking turns on the air

Only one station can talk on a radio channel at a time; two at once garble both (a “collision”). Before transmitting, every modem *listens first*: incoming audio energy is compared against an automatically learned background-noise level, and the channel counts as busy when audio is 4× louder than that background (held for 400 ms past the last loud moment so brief gaps inside someone’s transmission don’t look like free air). When the channel goes quiet, the station still waits a random 150–550 ms — so two stations that were both waiting don’t key up at the same instant. After 8 seconds of continuous busy (15 s on Mercury) the frame is sent anyway so traffic can never be blocked forever. AFSK also uses its demodulator’s data-carrier-detect as a second “someone is talking” signal.

4. Mesh Network Layer

Implements “Adaptation of Uncoordinated Distance-Vector Routing for Unencrypted Amateur Radio Networks” (see References), with one approved deviation: packet payloads carry compact HBC binary rather than ITA2 text.

In plain terms: every station keeps a simple address book (“to reach DAVE, hand the message to BOB”). The address book is built just by listening, and messages for far-away stations get passed station-to-station like a relay race.

4.1 Common Packet Header (13 bytes)

Every mesh packet starts with the same 13 bytes:

Byte offset	Length	Field	Description
0	1	Packet Type	0x01 Announce, 0x02 Broadcast, 0x03 Direct, 0x04 ACK
1	5	Origin Callsign	Who originally created the packet (bit-packed ITA2, section 4.2)
6	5	Transmitter Callsign	Who is transmitting this particular hop; each relay overwrites it with its own callsign
11	2	Sequence ID	A 16-bit counter value (big-endian) used to recognize duplicates (section 4.8)

4.2 Callsign Encoding (ITA2, 5 bytes)

Callsigns are stored in an old teletype alphabet called ITA2, which uses only 5 bits per character instead of 8 — so 8 characters fit in exactly 5 bytes (40 bits). Two special “shift” codes switch between letters (LTRS = 11111) and figures/digits (FIGS = 11011); unused positions are filled with NULL (00000). A leading LTRS is written before the first letter; if a callsign would need 9 codes (for example KE8TQB), the leading LTRS is dropped — decoders start in letters mode anyway, so both forms read back identically.

Example	Code sequence	Packed bytes (hex)
K1ABC	LTRS K FIGS 1 LTRS A B C	FF B7 DF E2 6E
N3DEF	LTRS N FIGS 3 LTRS D E F	F9 B7 0F CA 16
W2XYZ	LTRS W FIGS 2 LTRS X Y Z	FE 77 9F DE B1

NOTE: The source paper's K1ABC example lists byte 03 as BF, but the paper's own bit-by-bit math works out to DF; this implementation follows the math. All three vectors above are locked in by the automated test suite (codec-test/MeshRouterTest.java).

4.3 Packet Type 0x01 — Announce (14 bytes)

A tiny “I exist and here’s how far away I am” packet that builds everyone’s address book.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x01
13	1	Hop Count	How many relays the announce has passed through; the origin sets 0, each relay adds 1

4.4 Packet Type 0x02 — Broadcast (13 + n bytes)

A message to everyone in earshot. Not relayed, not acknowledged.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x02
13+	variable	Payload	One HBC message (section 5)

4.5 Packet Type 0x03 — Direct Message (23 + n bytes)

A message for one specific station, passed hop-by-hop.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x03
13	5	Destination Callsign	The final recipient
18	5	Next Hop Callsign	The neighbor being asked to carry this hop; each relay rewrites it
23+	variable	Payload	One HBC message

A station that sees its own callsign in Next Hop looks the Destination up in its address book, writes in the next station’s callsign, and re-transmits. The final destination delivers the payload

and answers with an ACK (section 4.6). If nobody knows a route, a relay drops the packet; the original sender instead falls back to sending it as a Broadcast.

4.6 Packet Type 0x04 — Acknowledgment (20 bytes)

The “got it!” receipt for a Direct message, sent back to the original sender.

Byte offset	Length	Field	Description
0–12	13	Common Header	Type = 0x04. Origin = the station acknowledging.
13	5	Destination Callsign	The Origin of the Direct being acknowledged
18	2	Acked Sequence ID	The Sequence ID of the Direct being confirmed

Stations along the way forward ACKs toward their destination when they know a route; the duplicate cache (section 4.8) keeps that from snowballing.

4.7 Route Discovery and Maintenance

- **Announce propagation** — when a station hears a new Announce, it writes the entry “Origin is reachable via whoever just transmitted this” and re-broadcasts the announce with hop count + 1. Shorter routes (fewer hops) replace longer ones; ties go to whichever route was heard first.
- **Passive learning** — hearing *any* valid frame is proof of a path, so the transmitting station is recorded as a direct neighbor, and (for non-announce frames) the originating station is recorded as reachable through that transmitter. Hearing a single position report is enough to be able to send to that station.
- **Announce-on-traffic** — a station’s own transmissions do the announcing for it; an explicit Announce packet only goes out after 10 quiet minutes, as a keepalive.
- **Expiration** — address-book entries are erased 30 minutes after the last frame that refreshed them (stations move and links change); a cleanup pass runs every 60 s.

4.8 Loop Prevention

Without protection, a rebroadcast message could bounce between stations forever. Each receiver therefore remembers [Origin + Sequence ID + Type] combinations for 5 minutes; a packet it has already seen is silently ignored. One deliberate exception: a duplicate Direct addressed *to this station* gets a fresh ACK (but is not delivered twice) — that covers the case where the first ACK was lost on the air.

4.9 Reliability and Retries

Directs work “fire and wait”: after sending, the originator waits 5 s (plus a random 0–2 s) for the ACK. No ACK → send it again **with the same Sequence ID**, up to 3 retries, then declare it failed and log it. Broadcasts are never acknowledged or retried.

5. Application Layer (HBC Protocol v1.6)

The payload inside every Broadcast/Direct is one HBC message: a map event squeezed into as few bytes as possible by packing values bit-by-bit instead of sending bulky XML text. (For scale: a normal ATAK position report is roughly 1–2 *thousand* bytes of XML; the HBC version is about 15 bytes.) Full field-level definitions and worked byte examples live in the HBC-Protocol repository (see References); this section summarizes the interface.

5.1 HBC Header

Field	Size	Description
Callsign	variable (ITA2, CR-terminated)	Sending operator/device callsign, max 8 characters
Version	3 bits	000 = version 1
Mode	3 bits	mode-1 for Modes 1–6; 111 = Mode 0 Ack

NOTE: HBC uses its own ITA2 table (bit-reversed relative to the mesh header’s table) and marks the end of text with a CR code instead of padding to a fixed size. The two layers’ callsign encodings are cousins, not twins — they are not interchangeable.

5.2 Message Modes and Sizes

Mode	CoT type(s)	What it carries	Typical size
0 Ack	b-t-f-d / b-t-f-r	Chat delivered/read receipt: recipient callsign, kind (2 bits), 16-bit tag	~12 B
1 PLI / Spot	a-f-G, a-h-G, a-n-G, a-u-G	Position report: spot flag, affiliation (2 bits), name (7 chars), latitude/longitude (21+22 bits)	~15 B
2 Alert	b-a-o-tbl / b-a-o-can	911 emergency: active flag, alert + originator names, position	~18 B
3 GeoChat	b-t-f	Text message: destination kind (All / Room / DM), room or recipient, DM 16-bit tag, ITA2 text	DM “helo landing zone” = 23 B
4 Shape	u-d-c-c / u-d-r / u-d-f	Drawing: circle (center + radius) or polygon/polyline (≤15 points)	varies
5 CASEVAC	b-r-f-h-c	Medical evacuation request (9-line summary fields)	~20 B
6 Extended Marker	any placed marker	Full CoT type token, icon reference (2525C / spot-map / custom), color tint	varies

5.3 Deterministic UUIDs

ATAK identifies every map object by a UUID (unique ID string). HBC does not transmit UUIDs — both ends compute the same one from the callsign, so a station's position marker updates in place instead of piling up duplicates:

Object	UUID rule
Position (PLI) marker	HBC-{CALLSIGN}
911 alert	HBC-{CALLSIGN}-911
DM conversation	Recipient's UUID; the receiving plugin swaps in its real ATAK device UUID so the message lands in the right chat window
Spots / shapes / markers	Random UUID each time one is received

5.4 Direct-Message Tags and Receipts

Every chat DM carries a 16-bit tag — a CRC-16/CCITT-FALSE checksum (polynomial 0x1021, initial value 0xFFFF) of ATAK's internal `messageId`. When the recipient's ATAK automatically generates its delivered/read receipts, the plugin converts them into tiny Mode 0 Acks that echo the tag back over the air; the sender matches the tag to the original `messageId` and injects the receipt, so ATAK's familiar delivered/read checkmarks work end-to-end over radio. These application-level receipts are separate from (and in addition to) the mesh-layer ACKs of section 4.6.

5.5 Contact Publication

ATAK only lists a station in its chat/contact pickers when that station's position report includes a network "endpoint." Received PLIs are therefore reconstructed with `<contact callsign="..." endpoint="*:-1:stcp"/>` and a `<__group>` team entry, so hearing a single position report makes the station selectable for chat and file/point sends. Outgoing traffic to that contact is intercepted by the plugin and transmitted over the mesh.

6. User Interface Reference (Screenshots)

All configuration lives in one pane, opened from the HBC toolbar icon. The figures below show each control described in this document.



Figure 1 — The Settings tab: link status (STOPPED) with the “Next PLI” countdown beneath it, ham callsign entry, the Send-to destination selector, and the TX Dwell / VOX Lead / PLI rate timing fields (sections 3.1, 3.4, 4.7).

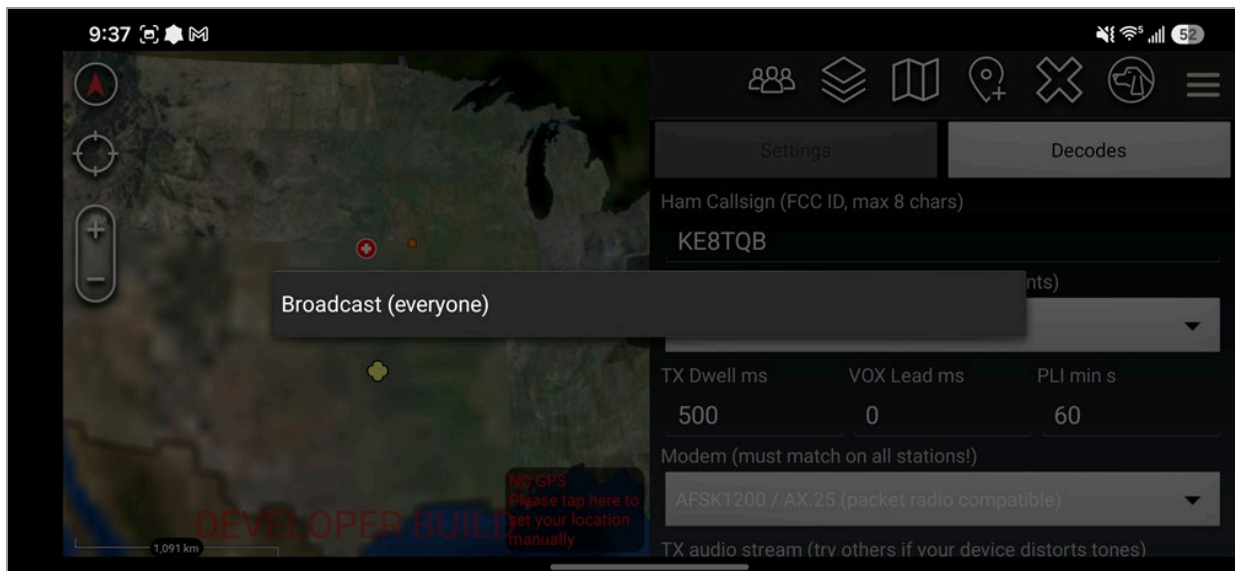


Figure 2 — The Send-to selector expanded. “Broadcast (everyone)” is the default; every station learned by the mesh (section 4.7) is added to this list automatically and can be chosen as a routed Direct destination for markers/points.

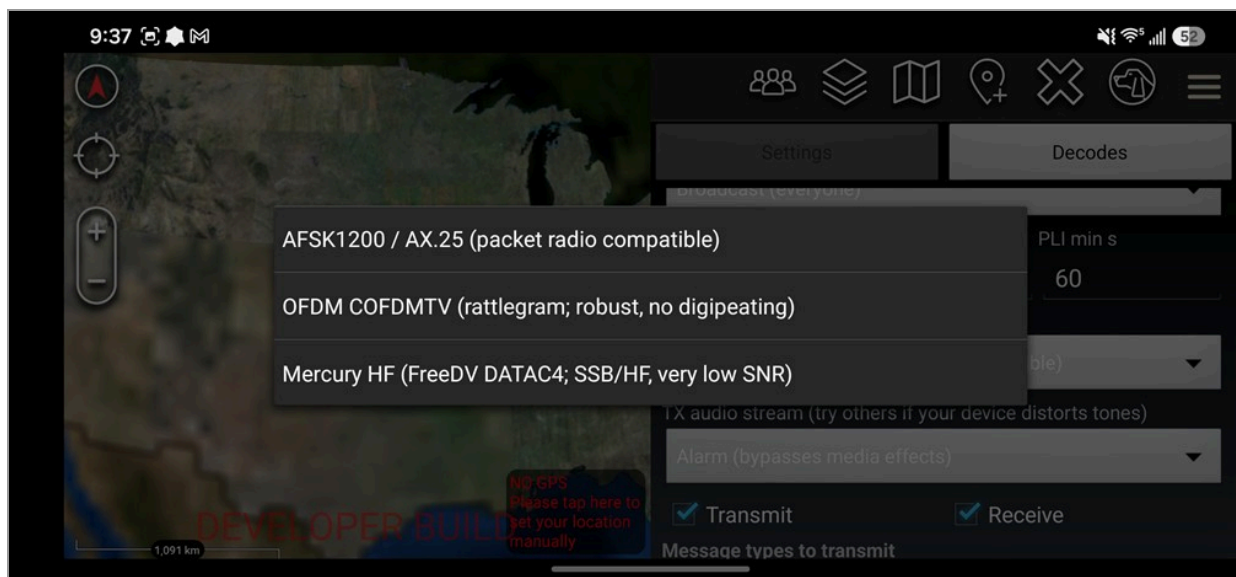


Figure 3 — Modem selection (section 3): AFSK1200/AX.25, OFDM COFDM TV, or Mercury HF (FreeDV DATA4). All stations on a channel must choose the same modem.

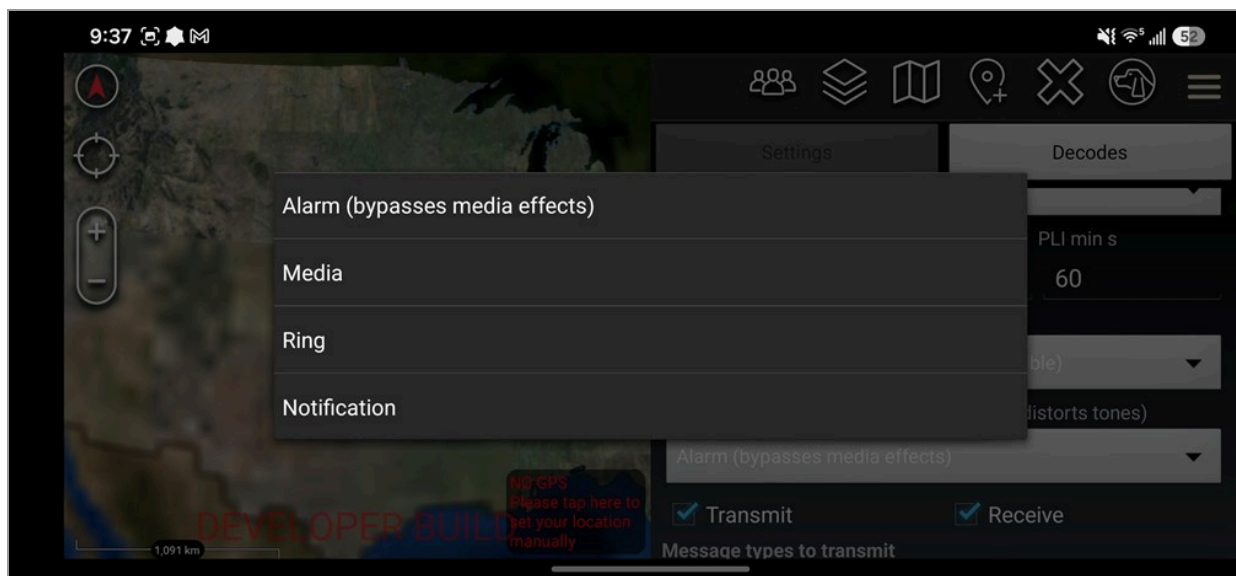


Figure 4 — TX audio stream selection. “Alarm” is the default because it bypasses phone media effects (EQ, spatial audio) that can distort modem tones; the alternatives exist for devices that behave differently.

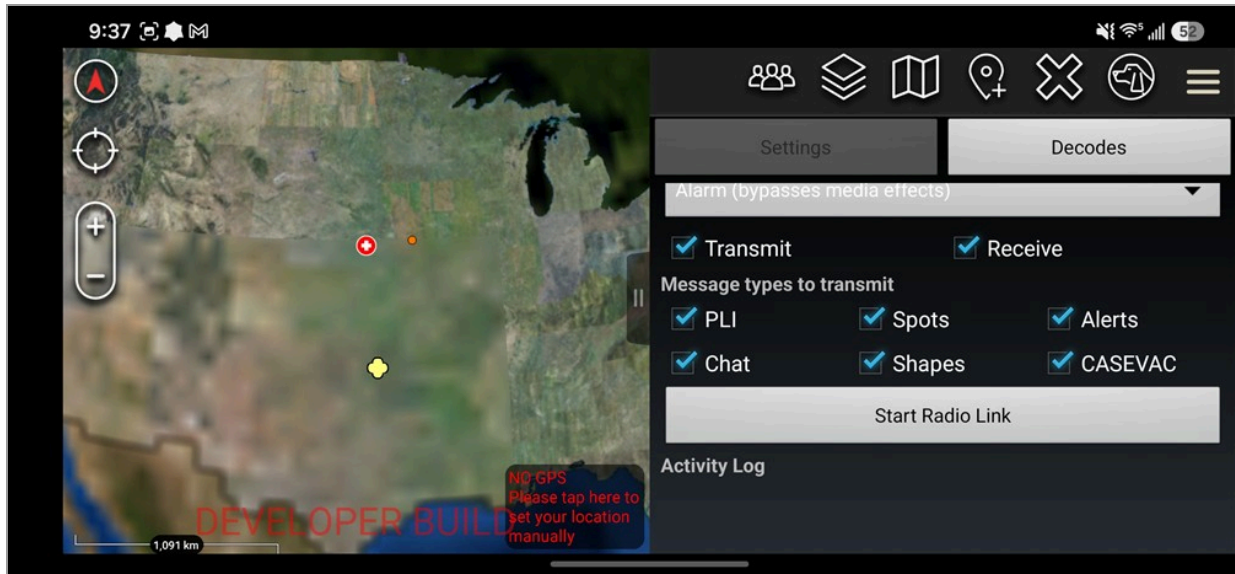


Figure 5 — Transmit/Receive master switches, the per-type transmit filters (PLI, Spots, Alerts, Chat, Shapes, CASEVAC — the HBC modes of section 5.2), the Start Radio Link button, and the Activity Log.

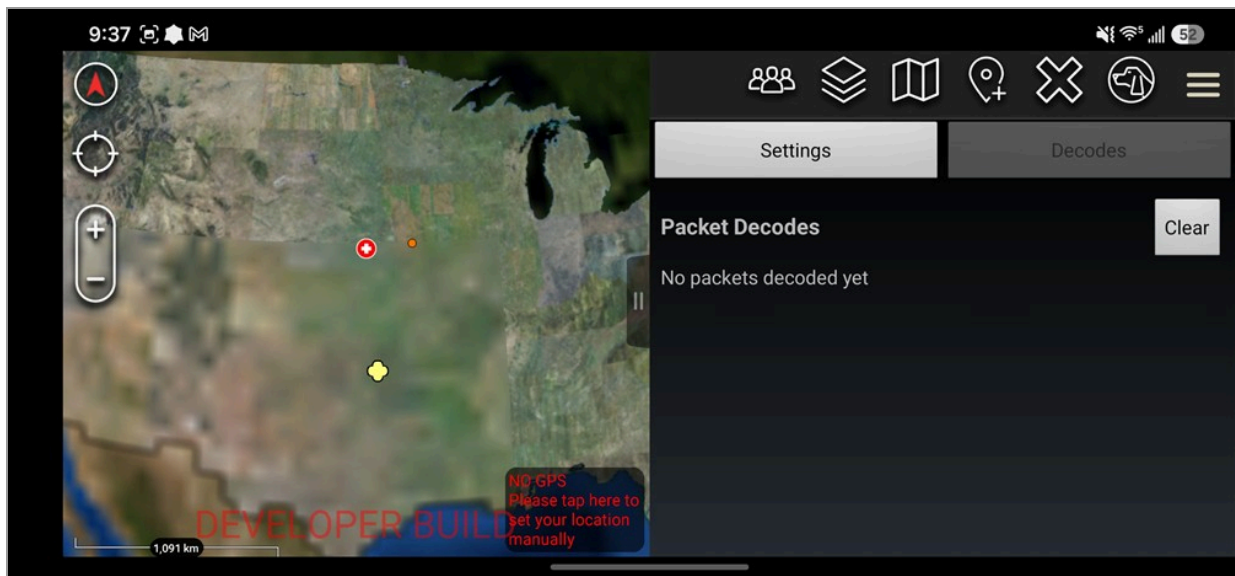


Figure 6 — The Decodes tab: a running log of every packet successfully received and decoded, with sender, mode, and payload size.

7. Timing Parameters and Constants

Parameter	Value	Layer
Route entry TTL	30 min (refreshed by any heard frame)	Mesh
Duplicate cache TTL	5 min	Mesh
Cleanup interval	60 s	Mesh

Direct retry timer	5 s + 0–2 s jitter, max 3 retries	Mesh
Quiet-time announce keepalive	10 min without own traffic	Mesh
CSMA busy threshold / hold	4× adaptive noise floor / 400 ms	PHY
CSMA backoff / give-up	150–550 ms clear / 8 s (15 s Mercury)	PHY
PLI rate limit	User setting, default 60 s (Figure 1)	Application
RX duplicate window (frame hash)	30 s	Plugin

8. Compatibility and Constraints

- All stations on a channel must run the same plugin wire version (0.12+ mesh framing) and select the same modem (Figure 3).
- Mercury DATAC4's small frame limits routed Directs to ~29 bytes of HBC payload (Broadcasts ~39 B). Every standard message fits; unusually large Mode 6 markers may need Broadcast or a different modem.
- Callsigns: at most 8 ITA2 characters (A–Z, 0–9, hyphen).
- No encryption is used or permitted; never transmit sensitive data.

9. Glossary

Term	Meaning
ATAK	Android Team Awareness Kit — the mapping app this plugin extends
CoT	Cursor on Target — the XML message format ATAK uses for map events
EUD	End User Device — the phone/tablet running ATAK
PLI	Position Location Information — a “here I am” report
Modem	Modulator/demodulator — converts bytes to sound and back
Frame / packet	One complete chunk of bytes sent over the air
Payload	The useful data inside a packet, after the addressing header
Hop	One radio link in a relay chain (A→B is one hop; A→B→C is two)
Route / next hop	Address-book entry: which neighbor to hand a message to so it reaches a destination
ACK	Acknowledgment — a short “received OK” reply
CSMA	Carrier Sense Multiple Access — listen before transmitting to avoid collisions
VOX	Voice-operated transmit — the radio keys up automatically when it hears audio

CRC	Cyclic Redundancy Check — a checksum that detects corrupted data
ITA2	A 5-bit teletype alphabet from the telegraph era; smaller than normal 8-bit text
TTL	Time To Live — how long a record is kept before being erased
UID	Unique identifier string for a map object in ATAK

10. References

1. HBC Protocol reference implementation and specification — github.com/ke8tqb/HBC-Protocol
2. HBC ATAK Plugin source — github.com/ke8tqb/HBC-ATAK-Plugin
3. *Adaptation of Uncoordinated Distance-Vector Routing for Unencrypted Amateur Radio Networks* (routing protocol paper)
4. Mercury HF modem — mercury.hermes.radio; FreeDV/codec2 — github.com/drowe67/codec2
5. rattlegram / COFDMTV — github.com/aicodix/rattlegram

Appendix A — GitHub Repository File Structure

The plugin source lives at github.com/ke8tqb/HBC-ATAK-Plugin. The map below shows where everything described in this document is implemented.

HBC-ATAK-Plugin/	
├─ app/	
│ └─ build.gradle	Plugin version, ATAK version, NDK pin, signing config
│ └─ proguard-gradle.txt	Release-build shrinker rules
│ └─ src/main/	
│ └─ AndroidManifest.xml	Plugin discovery entry required by ATAK
│ └─ java/	
│ └─ com/atakmap/android/hbc/	
│ └─ HbcEncoder.java	Application layer: CoT XML → HBC bytes (Section 5)
│ └─ HbcDecoder.java	Application layer: HBC bytes → CoT XML
│ └─ Ita2.java	HBC's 5-bit text alphabet (Section 5.1)
│ └─ BitReader.java	Bit-level read helper for the codec
│ └─ BitWriter.java	Bit-level write helper for the codec
│ └─ MeshRouter.java	Mesh network layer: packets, routing, retries, ACKs (Section 4)
│ └─ CsmaSense.java	Listen-before-transmit logic (Section 3.4)
│ └─ AudioModem.java	AFSK1200 modem (Section 3.1)
│ └─ OfdmModem.java	OFDM COFDMTV modem (Section 3.2)
│ └─ OfdmNative.java	Java ↔ C++ bridge for the OFDM library
│ └─ MercuryModem.java	Mercury HF modem (Section 3.3)
│ └─ MercuryNative.java	Java ↔ C bridge for the FreeDV library
│ └─ plugin/	
│ └─ HbcPlugin.java	The plugin itself: settings pane, toolbar icons, TX/RX plumbing, ATAK integration
│ └─ sivantoledo/ax25/...	Bundled javAX25 library (AFSK/AX.25, GPL)
│ └─ cpp/	
│ └─ CMakeLists.txt	Native build: libhbcofdm + libhbcm Mercury
│ └─ hbc-ofdm-jni.cpp	C++ glue for the OFDM modem
│ └─ *.hh	Vendored aicodix OFDM/DSP headers
│ └─ hbc-mercury-jni.cpp	C glue for the FreeDV DATAC modem
│ └─ hbc-mercury-compatible.c	Math shims for older Android versions
│ └─ mercury/	Vendored FreeDV/codec2 sources (from the Mercury project; LGPL – see THIRD-PARTY.md)
│ └─ res/	
│ └─ layout/main_layout.xml	The settings/decodes pane (Figures 1–6)
│ └─ values/strings.xml	All UI text and dropdown options
│ └─ drawable/	Icons, incl. ic_radio_on/off (status icon)
├─ codec-test/	
│ └─ HbcCodecTest.java	Byte-exact tests for the HBC codec (runs on a PC)
│ └─ MeshRouterTest.java	Byte-exact + behavior tests for the mesh layer
├─ docs/	
│ └─ HBC_ICD.html	This document (editable source)
│ └─ HBC_ICD.pdf	This document (rendered)
│ └─ images/	Figures 1–6 (app screenshots)
└─ gradle/, gradlew, gradlew.bat	Gradle build system wrapper

— settings.gradle, build.gradle	Project-level build configuration
— .takdev/	TAK SDK build-plugin support files
— CHANGELOG.md	Version-by-version history of changes
— THIRD-PARTY.md	Licenses of all bundled third-party code
— README.md	Project introduction and build instructions

Where to look first: the wire formats of Section 4 are all in `MeshRouter.java`; the message encodings of Section 5 are in `HbcEncoder.java/HbcDecoder.java`; anything you can tap in the app is in `HbcPlugin.java` plus `res/layout/main_layout.xml`. The codec-test/ programs run on an ordinary computer (no phone needed) and prove the byte formats match this document.